

IMLE-VLA: Fast Single-Step Action Generation for Vision-Language-Action Policies

Kian Hosseinkhani^{1*} Qinhe Peng² George Shramko¹ Mehran Aghabozorgi¹ Jianing Qian²
Tristan Engst¹ Alireza Moazeni¹ Dinesh Jayaraman² Ke Li^{1,3,4}

¹Simon Fraser University ²University of Pennsylvania

³Alberta Machine Intelligence Institute (Amii) ⁴Canada CIFAR AI Chair

*Corresponding author

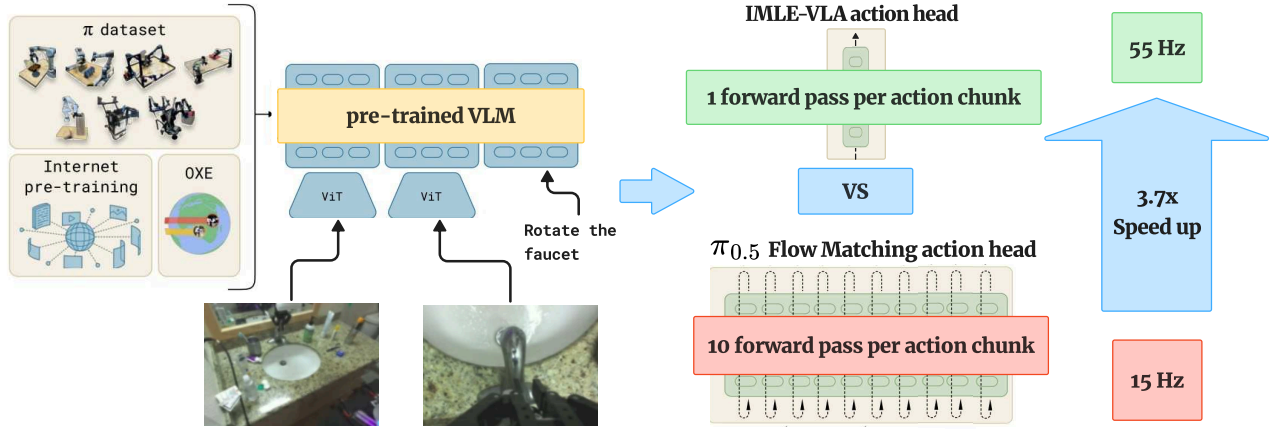


Fig. 1: Overview of IMLE-VLA: drop in single-step action head removing iterative modelling in VLAs

Abstract—Vision-language-action (VLA) policies leverage pretrained vision-language backbones to achieve strong cross-task generalization. A leading design couples this backbone with a dedicated continuous action head trained via diffusion or flow matching. However, such heads rely on iterative multi-step sampling, for example 10 Euler steps in $\pi_{0.5}$. This creates an inference bottleneck that produces stop-and-go movement in the robot and slower task completion. We introduce IMLE-VLA, which replaces the iterative action head with a single-step conditional generator trained via conditional Implicit Maximum Likelihood Estimation (cIMLE). The cIMLE objective promotes multimodal action coverage, avoiding the mode collapse of naive regression heads while eliminating multi-step sampling entirely. When IMLE-VLA is applied to $\pi_{0.5}$, it increases inference frequency $3.67\times$ (55 Hz vs. 15 Hz), enabling up to $11\times$ higher action throughput. On the 40-task LIBERO benchmark, IMLE-VLA achieves the highest average success rate (98.0%) among all baselines while leading in inference frequency. Under the test-time perturbations of LIBERO-plus, IMLE-VLA retains $\pi_{0.5}$'s robustness while other baselines degrade sharply, confirming that the cIMLE head preserves generalization. Real-world experiments on a Franka Emika Panda across four tasks demonstrate smoother motion ($2.2\times$ – $3.0\times$ lower jerk) and faster task completion, with IMLE-VLA outperforming $\pi_{0.5}$ on every task and reducing average VLA inference time per episode by $3.9\times$ – $6.6\times$. Videos and code are available at <https://kianhk6.github.io/IMLE-VLA/>.

I. INTRODUCTION

Vision-language-action (VLA) policies build on pretrained vision-language models (VLMs), inheriting internet-scale visual and linguistic knowledge that enables strong generalization across tasks and embodiments [1]–[4], making them the dominant generalist paradigm in robot learning. However, this generalization comes at a steep computational cost. Large vision-language backbones combined with iterative action-sampling procedures make VLA inference a critical bottleneck for high-frequency real-time robot control [5]–[12]. The resulting latency introduces dead time in the control loop: under synchronous execution, the robot stalls until the next prediction is ready, producing stop-and-go execution and longer task completion times.

This bottleneck primarily stems from the way VLAs generate actions, which follows one of two dominant paradigms: autoregressive decoding of discrete tokens and multi-step sampling of continuous actions. Autoregressive VLAs represent each action as a sequence of discrete tokens and decode them one at a time [2]–[4]. Furthermore, their reliance on discrete tokenization can limit the fine-grained continuous precision needed for dexterous manipulation [13], [14]. The second class generates continuous actions through multi-step sampling with diffusion or flow matching, typically

via an action expert attached to a VLM backbone such as $\pi_{0.5}$ [14]. Here the model predicts a chunk of future actions together, yet producing each chunk still requires multiple sequential forward passes (e.g., 10 Euler steps) [13]–[18]. Consequently, action generation remains a primary bottleneck for real-time control.

One of the key motivations for diffusion and flow-matching action heads is their capacity to handle the inherent multimodality of robot action distributions [13], [14]. A single instruction such as “clear the table” can be executed through many valid action sequences, and these models aim to capture this complexity. A naive solution would be to eliminate sequential generation via a single-step regression head. However, such models are prone to *mode collapse* [13], [19], [20]: the regression objective averages over valid modes, pulling the prediction toward a single mean behavior that degrades task performance when the action distribution is multimodal. We empirically validate this effect in Sec. IV-C.

This tension between expressivity and efficiency raises a central question: *can a single-step generator with a mode-covering objective replace iterative action generation while improving task performance?*

In this work, we introduce **IMLE-VLA**, a single-step action generation framework for VLAs that replaces multi-step sampling with a conditional Implicit Maximum Likelihood Estimation (cIMLE) [21]–[25] objective that explicitly optimizes for mode coverage.

We instantiate IMLE-VLA by replacing the 10-step flow-matching action head of $\pi_{0.5}$ [14] with our single-step generator. As the modification is confined to the action head, the full VLM backbone is preserved unchanged. This yields a $3.67\times$ increase in inference frequency (55 Hz vs. 15 Hz). Combined with an extended execution horizon H , which lets the policy replan less frequently, this compounds into up to $11.0\times$ higher action throughput (inference frequency $\times H$) [6]–[8], [11], [12]. This directly targets the “stop-and-go” control paradigm [3], [4], [6], [13]–[15], [26] that high VLA latency imposes: the robot idles while the model computes, producing visible pauses and stuttering motion. By cutting the time the robot waits on inference, IMLE-VLA reduces these pauses and reacts faster in dynamic settings, leading to faster task completion.

We evaluate IMLE-VLA in both simulation and the real world. On the **LIBERO simulation benchmark** [27] (40 tabletop manipulation tasks across four suites), IMLE-VLA achieves the highest average success rate (98.0%) among all compared methods. To probe whether these gains hold under distribution shift, we further evaluate IMLE-VLA on **LIBERO-plus** [28], which applies systematic test-time perturbations to the LIBERO tasks. Under these conditions, IMLE-VLA retains $\pi_{0.5}$ ’s robustness while all other baselines degrade sharply. In a **real-world setting** with a Franka Emika Panda, we test four tasks spanning single-step manipulation, multi-stage sequential reasoning, and reactivity to a dynamic environment. IMLE-VLA outperforms $\pi_{0.5}$ on every task while producing measurably smoother motion, reducing proprioceptive jerk by $2.2\times$ – $3.0\times$. Across both

settings, we measure the wall-clock time spent on VLA forward passes per episode, excluding robot execution and all other non-inference overhead. IMLE-VLA reduces this by up to $10.5\times$ in simulation and up to $6.6\times$ on the real robot.

II. RELATED WORK

Our work targets the sequential action-generation bottleneck in generalist VLAs. We first review complementary system-level optimization strategies, and then review distillation and teacher-free single-step methods.

System-level optimizations. Recent work [12] uses hardware-specific kernel tuning to accelerate inference, yet latency remains bounded by the sequential steps of diffusion-based heads. IMLE-VLA eliminates this bottleneck via single-step generation. These system-level gains are complementary to our algorithmic approach; by leveraging their efficient Triton kernels, we further amplify our inference frequency as demonstrated in Sec. IV.

Distillation methods. One line of work accelerates VLAs by training a faster *student* to imitate a pretrained *teacher*, which ties the student’s behavior to the teacher’s output distribution. Shallow- π [7] distills $\pi_{0.5}$ into a shallower, lower-capacity student, and the resulting speedup comes at a measurable cost in success rate relative to the teacher (Sec. IV). A separate thread distills a single-step student from a multi-step teacher [29]–[31], leaving the student with the biases of the teacher’s iterative sampler. Unlike distillation-based accelerators, IMLE-VLA’s action head has no teacher: it is fit directly to expert actions under a cIMLE objective. Its outputs are therefore not capped by a teacher’s performance, nor shaped by an iterative sampler’s biases.

Teacher-free single-step methods. A separate line of work sidesteps the teacher entirely, learning single-step action generation directly from expert data. Among VLAs, the prominent example is OpenVLA-OFT [8], which replaces the autoregressive decoding of OpenVLA with an L1 regression head. L1 regression is known to collapse to the conditional median under multimodal action distributions [13], [19], [20], and OpenVLA-OFT attributes L1’s viability to the capacity of OpenVLA’s 7B backbone [8]. This reliance on a 7B backbone leaves OpenVLA-OFT $5.3\times$ slower than IMLE-VLA despite IMLE-VLA’s smaller 3B backbone (Table II). Yet this backbone scale does not confer robustness under distribution shift: on LIBERO-plus, OpenVLA-OFT’s success rate degrades sharply under test-time perturbations (Fig. 2). IMLE-VLA, by contrast, remains robust under these perturbations, despite its smaller 3B backbone.

From specialists to a generalist. Across both lines of work above, prior efforts at single-step multimodal action generation have largely been confined to specialist policies [29], [30], [32], task-specific models that require expensive retraining and data collection for every new setting. Even in these narrow domains, mitigating mode collapse requires complex auxiliary regularizers that encourage distribution coverage [33], [34]. The cIMLE objective [21] instead enforces mode coverage *by construction*, but its application

in robotics has been limited to specialist regimes [31], [35]–[37] and thus inherits the same generalization bottleneck. IMLE-VLA is the first to adapt cIMLE to VLAs, achieving single-step action generation in a generalist policy that leverages internet-scale vision-language pretraining to generalize across tasks and embodiments.

III. METHOD

We present **IMLE-VLA**, a framework for single-step, multimodal action generation in vision-language-action (VLA) policies. Our approach eliminates the computational bottleneck of multi-step iterative steps by training the action head with a conditional Implicit Maximum Likelihood Estimation (cIMLE) objective. Below, we detail our architectural framework and the mathematical formulation of cIMLE. Specifically, we first describe the coupling of a frozen VLM backbone with a lightweight, single-step action generator (III-A); then, we formalize the cIMLE training objective and its unique properties for modeling multimodal robotic behaviors (III-B).

A. Single-Step VLA Architecture

We operate within the VLA paradigm by coupling a high-capacity, frozen VLM backbone with a lightweight action head [13]–[15], [17]. By fixing the VLM parameters, we retain the robust spatial and semantic features acquired during internet-scale pretraining. Since the VLM constitutes the vast majority of model parameters and remains frozen, training reduces to the action head alone, keeping the cost minimal.

At each timestep t , the observation o_t consists of the visual inputs and language instruction. The VLM encodes it into a set of latent embeddings $f_{\text{VLM}}(o_t)$, which condition the action generation process. Unlike standard iterative methods such as diffusion or flow-matching, which require multiple sequential denoising steps, our approach uses a single-step conditional generator G_θ . This generator directly maps Gaussian noise to multimodal action chunks, effectively mitigating a significant latency contributor in iterative VLA inference.

Formally, we represent the generator output as an action chunk $A \in \mathbb{R}^{C \times D}$, where C denotes the prediction horizon and D is the action-space dimensionality. To generate a candidate, we sample a latent noise vector $z \sim \mathcal{N}(0, \mathbf{I})$ of the same dimensionality as A . The generator G_θ then maps the VLM embedding $f_{\text{VLM}}(o_t)$ and noise z to a complete action chunk in a single forward pass:

$$\hat{A} = G_\theta(f_{\text{VLM}}(o_t), z). \quad (1)$$

By sampling different latent vectors z , G_θ can produce diverse action chunk candidates that capture the underlying conditional distribution $p(A | o_t)$. This stochasticity ensures the model avoids regressing to a single average outcome, preserving the expressivity required for complex behaviors. We initialize G_θ from a pretrained $\pi_{0.5}$ checkpoint and fine-tune the action head using the cIMLE objective (III-B) while the VLM backbone remains frozen.

B. Conditional IMLE (cIMLE)

The pretrained $\pi_{0.5}$ action head is trained via Conditional Flow Matching (CFM) [13], [38], which necessitates the numerical integration of a learned velocity field during inference. In practice, this requires multiple sequential forward passes, typically 10 Euler steps, to generate a single action chunk, introducing a considerable inference-time integration bottleneck.

To eliminate this bottleneck, we seek to generate each action chunk in a single forward pass without sacrificing the multimodality of the action distribution. As discussed in Sec. I, a single instruction can be executed through many valid action sequences, and each of these behaviors is a distinct mode of the conditional action distribution $p(A | o_t)$. Although each training pair $(o_t^{(i)}, A_{\text{gt}}^{(i)})$ captures only one of these behaviors, the data contains observations that are very similar yet correspond to very different ground-truth actions. To preserve this multimodality, a faster action head must therefore remain capable of assigning different actions to the same observation.

The conditional IMLE (cIMLE) objective [21] achieves exactly this: it reformulates action generation as a *single-step* mapping that transforms latent noise directly into action chunks, eliminating the iterative sampling cost while retaining the expressivity of the underlying distribution. Given a training set $\{(o_t^{(i)}, A_{\text{gt}}^{(i)})\}_{i=1}^n$, for each data point i we draw m (the *sample factor*) independent noise vectors $\{z_{i,1}, \dots, z_{i,m}\}$ and generate a set of m candidate action chunks $\{\hat{A}_{i,j}\}_{j=1}^m$, conditioned on the corresponding VLM embedding $f_{\text{VLM}}(o_t^{(i)})$:

$$\hat{A}_{i,j} = G_\theta\left(f_{\text{VLM}}(o_t^{(i)}), z_{i,j}\right), \quad j = 1, \dots, m. \quad (2)$$

The cIMLE training procedure proceeds in two alternating steps. First, we perform a gradient-free **assignment step** where, for each ground-truth action $A_{\text{gt}}^{(i)}$, we identify its nearest neighbour among its m candidates generated from the same observation $o_t^{(i)}$ (see Eq. 2):

$$j^*(i) = \arg \min_{j \in \{1, \dots, m\}} \left\| \hat{A}_{i,j} - A_{\text{gt}}^{(i)} \right\|_2^2. \quad (3)$$

Next, in the **update step**, we minimize the distance between each ground-truth action and its assigned nearest neighbour:

$$\mathcal{L}_{\text{cIMLE}} = \frac{1}{n} \sum_{i=1}^n \left\| \hat{A}_{i,j^*(i)} - A_{\text{gt}}^{(i)} \right\|_2^2. \quad (4)$$

To see why this objective preserves multimodality, consider two illustrative regimes for the sample factor m : one with $m=1$, and another with $m > 1$. Setting $m = 1$ makes the nearest-neighbour selection $j^*(i)$ trivial, reducing Eq. 4 to plain L2 regression. However, it is well-known that this objective is optimized by regression to the mean:

$$\mathcal{L}_{m=1} = \mathbb{E}_{(o_t, A_{\text{gt}})} \mathbb{E}_z \left\| G_\theta(f_{\text{VLM}}(o_t), z) - A_{\text{gt}} \right\|_2^2, \quad (5)$$

whose minimizer ignores the noise and outputs the conditional mean:

$$G_{\theta^*}(f_{\text{VLM}}(o_t), z) = \mathbb{E}[A_{\text{gt}} | o_t] \quad \text{for all } z. \quad (6)$$

In effect, even when there are multiple correct actions for a given observation, this model is still optimized by averaging the multiple correct actions into one. Yet this average can lie between the modes, typically matching none of the demonstrated behaviors, making it an invalid action.

The assignment step in Eq. 3 is what changes this. With $m > 1$ candidates, each ground-truth action is matched to only its nearest candidate, allowing the other candidates to cover other modes without penalty by the cIMLE objective. Together, the outer sum ensures every mode is covered, while the nearest-neighbour assignment lets the generator map different latent codes z to different modes rather than collapsing them into a single averaged action.

Algorithm 1 cIMLE Training

Input: Training batch $\{(o_t^{(i)}, A_{\text{gt}}^{(i)})\}_{i=1}^B$, number of samples m , frozen VLM f_{VLM} , action head G_θ

Output: Updated parameters θ

// Preprocessing: Compute VLM embeddings once per batch

Compute $f_{\text{VLM}}(o_t^{(i)})$ once for all $i \in [B]$

// Step 1: Efficient Nearest Neighbour Search (Gradient-free)

for $i = 1$ **to** B **do**

 Sample m noise vectors $\{z_{i,j}\}_{j=1}^m \sim \mathcal{N}(0, I)$

 Generate m candidates $\hat{A}_{i,j} = G_\theta(f_{\text{VLM}}(o_t^{(i)}), z_{i,j})$,

$j \in [m]$

$j^*(i) \leftarrow \arg \min_{j \in [m]} \|\hat{A}_{i,j} - A_{\text{gt}}^{(i)}\|_2^2$

end for

// Step 2: Training via Assigned Nearest Neighbours

$\hat{A}_i^* \leftarrow G_\theta(f_{\text{VLM}}(o_t^{(i)}), z_{i,j^*(i)})$ for all $i \in [B]$ $\triangleright$ *With gradients*

$\mathcal{L} \leftarrow \frac{1}{B} \sum_{i=1}^B \|\hat{A}_i^* - A_{\text{gt}}^{(i)}\|_2^2$

$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ $\triangleright$ *Update action head only*

As detailed in Algorithm 1, this procedure adds negligible training overhead. While the training procedure is presented sequentially for conceptual clarity, the assignment step (Step 1) is implemented in parallel across both the batch and the m samples, making it highly efficient. In practice, we find that $m=2$ strikes an effective balance between expressivity and computational efficiency; we ablate this choice in Sec. IV-C. The VLM embedding $f_{\text{VLM}}(o_t^{(i)})$ is computed once per sample and shared across all m candidates, avoiding redundant backbone passes. Furthermore, the selection of $j^*(i)$ is performed without gradient tracking; only the winning candidate is recomputed for the backward pass. Consequently, cIMLE acts as a lightweight objective that transforms a multi-step VLA into a highly efficient single-step generator while explicitly optimizing for mode coverage, yielding substantial gains in inference frequency without sacrificing the policy’s capacity to handle complex, multimodal tasks.

IV. EXPERIMENTS

In this section, we evaluate the effectiveness of IMLE-VLA through a comprehensive set of evaluations in simulation and on a real-world robot. Our goal is to demonstrate that single-step action generation via cIMLE can achieve significant gains in inference frequency while maintaining or even improving the success rates of VLA policies.

All experiments use a standard receding-horizon loop: the policy observes o_t , runs one forward pass to predict an action chunk (Sec. III-A), and the robot executes the first H actions of that chunk *open-loop* before observing again and repeating. Here H is the *execution horizon*, a fixed number of actions executed per forward pass, so a larger H replans less often. We report three metrics: *task success rate*, our primary measure of policy quality; *action throughput*, the number of executable actions delivered to the robot per second, equal to inference frequency $\times H$; and *VLA-only wall-clock* (VLA-WC), the total time spent exclusively on VLA forward passes per successful episode, excluding robot execution and other non-inference overhead.

We benchmark IMLE-VLA against the $\pi_{0.5}$ flow-matching baseline across four primary axes:

- (i) Inference frequency, where we characterize raw inference frequency and action throughput;
- (ii) Simulated performance on the LIBERO benchmark, comprising 40 diverse manipulation tasks;
- (iii) Robustness under test-time distribution shift, evaluated on LIBERO-plus;
- (iv) Real-world experiments on four tasks assessing single-step manipulation, multi-step reasoning and reactivity;

A. Inference frequency

Deployment on physical robots imposes strict constraints on *inference frequency*: the number of VLA forward passes completed per second, directly governing the replanning rate. Higher inference frequency enables tighter control loops, smoother motion and faster response to new visual stimuli.

We benchmark efficiency on a single NVIDIA L40S GPU using natural task-specific language prompts and the standard two-view image inputs. Our protocol reflects deployment-realistic conditions: unlike prior benchmarks using empty prompts or static buffers [12], we include the mandatory cost of tokenizing the language instruction and proprioceptive state at every forward call.

Table I compares IMLE-VLA against several optimized implementations of the $\pi_{0.5}$ [14] VLA. The canonical JAX implementation runs at 15 Hz. Porting to PyTorch with `torch.compile` [39] and dedicated Triton kernels [12] yield modest gains, reaching 20 and 25 Hz respectively. This is because the bottleneck is algorithmic: the 10-step flow-matching loop imposes a fundamental ceiling that system-level optimizations alone cannot overcome. High latency is particularly impactful in physical deployments, where slow inference cycles manifest as noticeable “stop-and-go” pauses and reduced reactivity to environmental changes (see Sec. IV-D for concrete examples).

IMLE-VLA sidesteps this bottleneck by replacing the iterative action head with a single-step conditional generator while maintaining the exact same VLM backbone architecture. This yields an inference frequency of 55 Hz, a $3.67\times$ increase over the canonical JAX baseline, in turn enabling smoother robotic motion and more responsive control without reducing model capacity or backbone scale.

System	Inf. Freq. (Hz) $\uparrow$	Inf. Freq. ($\times$) $\uparrow$
$\pi_{0.5}$ (original JAX)	15	$1.00\times$
PyTorch + compile	20	$1.33\times$
Triton	25	$1.67\times$
IMLE-VLA (Ours)	55	$3.67\times$

TABLE I: **Inference frequency** measured on an NVIDIA L40S, the number of VLA forward passes per second, averaged over 50 episodes across 10 LIBERO-Long tasks. IMLE-VLA achieves 55 Hz, a $3.67\times$ increase.

B. Simulation: LIBERO Benchmark

We next characterize the trade-off between inference frequency and task success on the LIBERO benchmark [27], which comprises 40 tabletop manipulation tasks across four suites (Spatial, Object, Goal, and Long) with a Franka Emika Panda arm. Each suite contains 10 tasks; every task is evaluated over 50 episodes following the standard protocol.

Success rate comparison. Table II compares IMLE-VLA against iterative action heads [13], [14] and speed-up strategies surveyed in Sec. II. Inf. ($\times$) denotes the inference-frequency ratio relative to $\pi_{0.5}$ on matched hardware. We measure $\pi_{0.5}$, IMLE-VLA, and OpenVLA-OFT ourselves on an L40S; ratios for the remaining baselines are taken from Jeon et al. [7], who measure on H100.

With $H=10$, IMLE-VLA achieves **98.0%** average success, the highest among all compared methods, while also leading in inference frequency at $3.67\times \pi_{0.5}$. The fastest competing accelerator, Shallow- $\pi_{0.5}$ -L6 ($2.3\times$), is still slower than IMLE-VLA and drops to 94.5% due to its reduced model capacity [7]. OpenVLA-OFT reaches 94.5% on the same unfiltered dataset and runs below $\pi_{0.5}$ in inference frequency ($0.69\times$), reflecting the cost of its 7B-parameter backbone. IMLE-VLA is the only method that simultaneously leads in both success rate and inference frequency.

Robustness under distribution shift. To verify that our gains generalize beyond LIBERO’s training distribution, we additionally evaluate on **LIBERO-plus** [28], which applies systematic test-time perturbations to the LIBERO tasks across four axes (background, robot initial state, language, layout) at five severity levels (L1–L5). Across all axes and severities, IMLE-VLA retains $\pi_{0.5}$ ’s robustness, while all other baselines exhibit larger drops in success rate as severity increases (Fig. 2). Consistent with the L1-regression limitation discussed in Sec. II [8], OpenVLA-OFT’s success rate degrades sharply. IMLE-VLA’s cIMLE objective instead enforces mode coverage *by construction* [21], avoiding this failure mode.

Action throughput. Table III reports action throughput

(inference frequency $\times H$) across configurations. At $H=10$, IMLE-VLA already delivers $3.67\times$ more actions per second than the $\pi_{0.5}$ baseline, purely from replacing the flow-matching head. At $H=30$, the single-step inference advantage compounds with the longer horizon, yielding $11.0\times$ higher throughput.

VLA-only wall-clock (VLA-WC). Table IV reports VLA-WC across horizons, averaged over successful LIBERO episodes. At $H=10$ the number of forward passes per episode is nearly identical to $\pi_{0.5}$, yet VLA-WC drops by $3.6\times$ due to the lighter single-step action head. At $H=30$ the forward-pass count itself falls by $\sim 3\times$; combined with the faster per-pass inference, this yields an overall $10.5\times$ reduction.

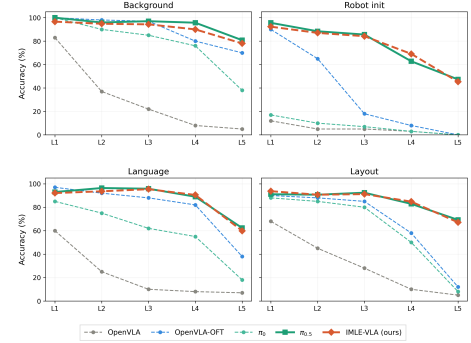


Fig. 2: **LIBERO-plus robustness under distribution shift.** Success across four perturbation dimensions, each at five difficulty levels (L1–L5). Unlike baselines, which degrade sharply as severity increases, IMLE-VLA matches $\pi_{0.5}$ ’s robustness throughout.

System	H	Action Throughput (Hz)	($\times$)
$\pi_{0.5}$ (JAX)	10	150	$1.00\times$
PyTorch + torch.compile	10	200	$1.33\times$
Triton	10	250	$1.67\times$
Ours	10	550	$3.67\times$
Ours	30	1,650	$11.0\times$

TABLE III: **Action throughput** in LIBERO simulation, defined as inference frequency $\times$ execution horizon H . ($\times$) denotes the throughput ratio relative to $\pi_{0.5}$ at $H=10$. At $H=10$, IMLE-VLA delivers $3.67\times$ higher throughput purely from single-step generation; at $H=30$ this compounds to $11.0\times$.

System	Fwd. Passes	VLA-WC (s)	VLA-WC ($\times$)
$\pi_{0.5}$ (JAX, $H=10$)	15.4	1.03	$1.0\times$
Ours ($H=10$)	15.5	0.28	$3.6\times$
Ours ($H=30$)	5.4	0.10	$10.5\times$

TABLE IV: **VLA-only wall-clock (VLA-WC).** Wall-clock on VLA forward passes per successful episode (see Sec. IV). *Fwd. Passes*: average forward passes per episode. At $H=10$, IMLE-VLA reduces VLA-WC by $3.6\times$; at $H=30$ fewer passes compound with faster inference for $10.5\times$ overall.

C. Method Analysis

Horizon ablation. Each replan cycle consists of one forward pass followed by H open-loop actions. Because IMLE-VLA’s forward pass is $3.67\times$ faster, the saved inference

Model	H	Spatial	Object	Goal	Long	Avg $\uparrow$	Inf. ($\times$) $\uparrow$
$\pi_{0.5}$ [14]	10	97.2	<u>99.0</u>	<u>97.8</u>	96.0	<u>97.5</u>	1.0
$\pi_{0.5}$ [14]	30	95.0	98.0	96.2	95.2	96.1	1.0
π_0 [13]	–	96.8	98.8	95.8	85.2	94.2	1.12
OpenVLA-OFT [8]	–	95.2	94.2	95.2	93.2	94.5	0.69
CogVLA [40]	–	99.0	<u>99.0</u>	97.0	95.0	<u>97.5</u>	0.8
SmolVLA [41]	–	90.0	96.0	92.0	71.0	87.3	1.0
Shallow- $\pi_{0.5}$ -L9 [7]	–	99.0	98.0	97.0	93.0	96.8	1.7
Shallow- $\pi_{0.5}$ -L6 [7]	–	<u>98.0</u>	96.0	94.0	90.0	94.5	<u>2.3</u>
IMLE-VLA	10	<u>98.0</u>	99.8	98.2	96.0	98.0	3.67
IMLE-VLA	30	97.2	<u>99.0</u>	96.2	<u>95.8</u>	97.1	3.67

TABLE II: **LIBERO benchmark.** Success rates (SR, %) on four LIBERO suites (50 episodes per task) and their average. H denotes the execution horizon (actions executed open-loop per forward pass; “–” = not specified). Inf. ($\times$) is the inference frequency ratio relative to $\pi_{0.5}$. Best in **bold**, second-best underlined. IMLE-VLA achieves the highest average success rate (98.0% at $H=10$) while simultaneously leading in inference frequency ($3.67\times$).

time can be reinvested into a longer horizon without exceeding $\pi_{0.5}$ ’s cycle time, making larger horizons practical. However, extending H still trades *reactivity* for *throughput*: the robot executes more actions before conditioning on a new observation. Figure 3 reports IMLE-VLA’s average LIBERO success across all four suites at varying horizons. At $H=10$ the policy achieves 98.0% average success, and performance degrades gradually as H grows. $H=30$ offers the best throughput–accuracy balance, still reaching 97.1% while delivering $11.0\times$ higher action throughput (Table III), and outperforming $\pi_{0.5}$ at the same horizon (96.1%).

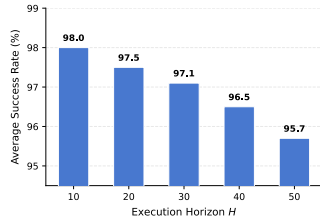


Fig. 3: **LIBERO horizon ablation.** IMLE-VLA success averaged across all four LIBERO suites at varying execution horizons. $H=10$ achieves 98.0%; larger horizons reduce reactivity and degrade success, while $H=30$ offers the best throughput–accuracy balance.

cIMLE sample factor ablation. The sample factor m controls how many candidate action chunks are generated per training example during the cIMLE assignment step (Eq. 2). When $m=1$ only one candidate exists, so the assignment is trivial and cIMLE reduces to standard L2 regression with no mode-covering guarantee.

Figure 4 reports LIBERO success across all four suites at $H=10$ for $m \in \{1, 2, 5\}$. Setting $m=1$ produces a clear performance drop, confirming that a pure regression objective is insufficient for modeling complex action distributions, an observation consistent with the limitation acknowledged by OpenVLA-OFT [8] (see Sec. II). With $m=2$ the mode-covering property activates and performance recovers substantially; increasing to $m=5$ yields nearly iden-

tical results, indicating diminishing returns. We therefore use $m=2$ throughout, as it achieves strong performance while minimizing the per-step training cost of generating additional candidates.

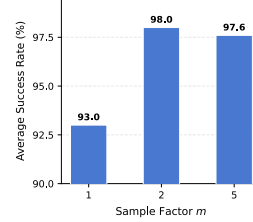


Fig. 4: **cIMLE sample factor ablation.** LIBERO success averaged across all four suites at $H=10$ for varying sample factor m . $m=1$ reduces cIMLE to standard regression (no mode-covering) and drops sharply; $m=2$ and $m=5$ are nearly identical, so we use $m=2$.

D. Real-world experiments

We evaluate IMLE-VLA against the $\pi_{0.5}$ baseline (canonical JAX implementation) on a Franka Emika Panda robot with an NVIDIA A6000 GPU, using two camera views (wrist and scene). Both policies are trained on DROID [42] and deployed zero-shot on four tabletop manipulation tasks (Figure 5), with success (task completion) reported over 20 episodes per task. $\pi_{0.5}$ runs at 15 Hz control frequency, while IMLE-VLA operates at 55 Hz. Unlike LIBERO’s deterministic contact dynamics, real-world manipulation introduces physical uncertainty (e.g. grasp slippage, surface friction variation), requiring more frequent replanning. We adopt the $\pi_{0.5}$ default horizon ($H=8$) and extend IMLE-VLA to $H=12$, since the faster inference compensates for the longer horizon while still yielding a shorter wall-clock replan interval than $\pi_{0.5}$. IMLE-VLA outperforms $\pi_{0.5}$ in every task while reducing the VLA-only wall-clock by $3.9\times$ – $6.6\times$ (Table V) and completing episodes approximately $2\times$ faster end-to-end (supplementary videos). The four tasks span single-step manipulation, multi-stage sequential reasoning,

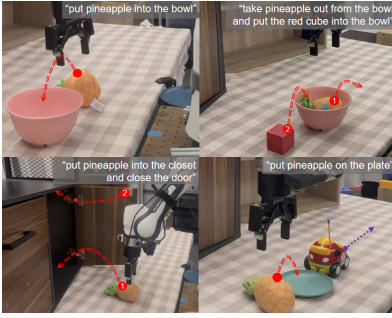


Fig. 5: **Real-world experiments.** We evaluate on three categories: **Single-Step Manipulation**, **Multi-Step Sequential Reasoning** and **Dynamic Reactivity**. IMLE-VLA generates faster, smoother, and more successful actions across all three categories.

Task	Success $\uparrow$		VLA-WC (s) $\downarrow$		Jerk ($\times$)
	Ours	$\pi_{0.5}$	Ours	$\pi_{0.5}$	
Pineapple in bowl	19	15	18.56	123	$2.7\times$
Swap pineapple & cube	18	15	31.1	178.3	$2.2\times$
Pineapple in cabinet	15	12	99.4	389.4	$3.0\times$
Pineapple on moving plate	16	12	16.7	89.5	$2.7\times$

TABLE V: **Real-world experiment results** on a Franka Emika Panda with an NVIDIA A6000 (20 episodes per task). Success: successful episodes out of 20. VLA-WC: wall-clock on VLA forward passes per successful episode, averaged over successful episodes per task. Jerk ($\times$): ratio of $\pi_{0.5}$'s mean proprioceptive jerk to ours (higher means smoother motion relative to the baseline; see Sec. IV-D). IMLE-VLA outperforms $\pi_{0.5}$ on every task, reduces VLA-WC by $3.9\times$ – $6.6\times$, and produces $2.2\times$ – $3.0\times$ lower jerk.

and reactivity to a dynamic environment.

- **Pineapple in bowl** (single-step): the robot grasps a squishy pineapple toy from the table and places it into a bowl.
- **Swap pineapple and cube** (multi-step): a bowl contains a squishy pineapple with a red cube beside it. The robot must remove the pineapple from the bowl, then place the red cube inside.
- **Put pineapple in cabinet** (multi-step): a cabinet door is already open with a squishy pineapple in front of it. The robot must pick up the pineapple, place it inside the cabinet, and close the door.
- **Pineapple on moving plate** (reactive): a remote-controlled car drives an attached plate across the table; the robot must place the pineapple on the moving plate, requiring continuous target adjustment.

Motion quality. IMLE-VLA produces markedly smoother motion than $\pi_{0.5}$: its faster inference minimizes the time the robot idles while the model computes, and its longer execution horizon means fewer replans, together eliminating the visible stop-wait-execute pauses and abrupt directional corrections that $\pi_{0.5}$ exhibits at each replan. To quantify this, we measure *proprioceptive jerk*, the third backward finite difference of the measured joint positions (7 arm joints,

gripper excluded) normalized by dt^3 , averaged over joints and steps; IMLE-VLA reduces mean jerk by $2.2\times$ – $3.0\times$ across all four tasks (Table V). The difference is apparent in the supplementary videos. On *swap pineapple and cube*, $\pi_{0.5}$ changes direction repeatedly before each placement, while IMLE-VLA follows a direct trajectory and finishes faster. The shorter replan interval also enables recovery when something goes wrong: on *pineapple in cabinet*, when the pineapple slips from IMLE-VLA's gripper during insertion, the robot quickly detects the failure and re-grasps. Meanwhile, $\pi_{0.5}$'s slower replanning introduces frequent abrupt corrections that significantly slow task completion.

Reactive task. The dynamic task (*pineapple on moving plate*) evaluates the ability to react to a continuously changing environment. In this setting, IMLE-VLA significantly outperforms $\pi_{0.5}$. Although IMLE-VLA executes a longer action horizon, its faster inference leads to a much shorter wall-clock replanning cycle. As a result, the policy effectively closes the perception–action loop more frequently in real time, allowing the robot to continuously track and adapt to the plate's motion.

This distinction is particularly important in dynamic scenes, where delays between perception and action quickly degrade performance. By minimizing idle time between replans, IMLE-VLA reduces the latency between observing the plate and responding to its updated position, enabling smoother interception and grasp attempts. In contrast, $\pi_{0.5}$ replans more slowly and therefore acts on increasingly stale observations. By the time a new action sequence is generated, the plate has already moved, causing the robot to repeatedly reach toward outdated target locations and ultimately fail to complete the task.

V. CONCLUSION

We presented **IMLE-VLA**, a framework that replaces the iterative flow-matching action head of VLAs with a single-step conditional generator trained via the cIMLE objective. By eliminating multi-step sampling while explicitly optimizing for mode coverage, IMLE-VLA achieves a $3.67\times$ increase in inference frequency and up to $11.0\times$ higher action throughput over the $\pi_{0.5}$ baseline. On the LIBERO benchmark, IMLE-VLA attains the highest average success rate (98.0%) among all compared methods while simultaneously leading in inference frequency. In real-world experiments on a Franka Emika Panda, the higher inference frequency produces smoother motion and reduces VLA-only wall-clock by $3.9\times$ – $6.6\times$, while outperforming $\pi_{0.5}$ on every task. Because our approach modifies only the action head, it serves as a drop-in replacement for the iterative heads in existing VLAs, a promising direction toward even faster generalist robot policies.

ACKNOWLEDGMENT

This research was enabled in part by NSERC, the Canada Foundation for Innovation, the Canada CIFAR AI Chairs program, the BC DRI Group and the Digital Research Alliance of Canada. The first author dedicates this work to the memory of his beloved grandfather.

REFERENCES

- [1] S. Reed *et al.*, “A generalist agent,” *ArXiv*, vol. abs/2205.06175, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:248722148>
- [2] Open X-Embodiment Collaboration, “Open x-embodiment: Robotic learning datasets and rt-x models,” *ArXiv*, vol. abs/2310.08864, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:264884565>
- [3] A. Brohan *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *ArXiv*, vol. abs/2307.15818, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260293142>
- [4] M. J. Kim *et al.*, “Openvla: An open-source vision-language-action model,” *ArXiv*, vol. abs/2406.09246, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:270440391>
- [5] W. Jiang *et al.*, “How fast can i run my vla? demystifying vla inference performance with vla-perf,” *ArXiv*, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285872613>
- [6] K. Pertsch *et al.*, “Fast: Efficient action tokenization for vision-language-action models,” in *Proc. Robotics: Science and Systems (RSS)*, 2025, arXiv:2501.09747. [Online]. Available: <https://api.semanticscholar.org/CorpusID:275570494>
- [7] B. Jeon *et al.*, “Shallow- π : Knowledge distillation for flow-based vlars,” *ArXiv*, vol. abs/2601.20262, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285102369>
- [8] M. J. Kim *et al.*, “Fine-tuning vision-language-action models: Optimizing speed and success,” *ArXiv*, vol. abs/2502.19645, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:276647709>
- [9] J. Zhang *et al.*, “Quantvla: Scale-calibrated post-training quantization for vision-language-action models,” *ArXiv*, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:286001346>
- [10] Y. Chen *et al.*, “Rlrc: Reinforcement learning-based recovery for compressed vision-language-action models,” *ArXiv*, vol. abs/2506.17639, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:280000516>
- [11] Y. Jiang *et al.*, “Asyncvla: Asynchronous flow matching for vision-language-action models,” *ArXiv*, vol. abs/2511.14148, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:283080185>
- [12] Y. Ma *et al.*, “Running vlars at real-time speed,” *ArXiv*, vol. abs/2510.26742, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:282591986>
- [13] K. Black *et al.*, “ $\pi 0$: A vision-language-action flow model for general robot control,” *ArXiv*, vol. abs/2410.24164, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:273811174>
- [14] Physical Intelligence *et al.*, “ $\pi 0.5$: a vision-language-action model with open-world generalization,” *ArXiv*, vol. abs/2504.16054, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:277993634>
- [15] Octo Model Team *et al.*, “Octo: An open-source generalist robot policy,” *ArXiv*, vol. abs/2405.12213, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:266379116>
- [16] S. Liu *et al.*, “Rdt-1b: a diffusion foundation model for bimanual manipulation,” *ArXiv*, vol. abs/2410.07864, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:273233386>
- [17] Q. Li *et al.*, “Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *ArXiv*, vol. abs/2411.19650, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:274423241>
- [18] L. Wang *et al.*, “Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers,” *ArXiv*, vol. abs/2409.20537, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:272986946>
- [19] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, pp. 1684 – 1704, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:257378658>
- [20] N. M. M. Shafiuallah *et al.*, “Behavior transformers: Cloning k modes with one stone,” *ArXiv*, vol. abs/2206.11251, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:249926747>
- [21] K. Li *et al.*, “Implicit maximum likelihood estimation,” *ArXiv*, vol. abs/1809.09087, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:52816401>
- [22] M. Aghabozorgi *et al.*, “Adaptive imle for few-shot pretraining-free generative modelling,” in *International Conference on Machine Learning*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260810286>
- [23] S. Peng *et al.*, “Chimle: Conditional hierarchical imle for multimodal conditional image synthesis,” vol. abs/2211.14286, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:254018031>
- [24] C. Vashist *et al.*, “Rejection sampling imle: Designing priors for better few-shot image synthesis,” in *European Conference on Computer Vision*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:272911336>
- [25] M. Aghabozorgi *et al.*, “Wimle: Uncertainty-aware world models with imle for sample-efficient continuous control,” *ArXiv*, vol. abs/2602.14351, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285616888>
- [26] T. Zhao *et al.*, “Learning fine-grained bimanual manipulation with low-cost hardware,” *ArXiv*, vol. abs/2304.13705, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258331658>
- [27] B. Liu *et al.*, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *ArXiv*, vol. abs/2306.03310, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259089508>
- [28] S. Fei *et al.*, “Libero-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.
- [29] A. Prasad *et al.*, “Consistency policy: Accelerated visuomotor policies via consistency distillation,” in *Proc. Robotics: Science and Systems (RSS)*, 2024, arXiv:2405.07503. [Online]. Available: <https://api.semanticscholar.org/CorpusID:269758119>
- [30] Z. Wang *et al.*, “One-step diffusion policy: Fast visuomotor policies via diffusion distillation,” *ArXiv*, vol. abs/2410.21257, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:273654186>
- [31] J. Dong *et al.*, “From flow to one step: Real-time multi-modal trajectory policies via implicit maximum likelihood estimation-based distribution distillation,” *ArXiv*, vol. abs/2603.09415, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:286429211>
- [32] K. Frans *et al.*, “One step diffusion via shortcut models,” *ArXiv*, vol. abs/2410.12557, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:273375140>
- [33] G. Zou *et al.*, “Dm1: Meanflow with dispersive regularization for 1-step robotic manipulation,” *ArXiv*, vol. abs/2510.07865, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:281950800>
- [34] —, “One step is enough: Dispersive meanflow policy optimization,” *ArXiv*, vol. abs/2601.20701, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285102124>
- [35] K. Rana *et al.*, “Imle policy: Fast and sample efficient visuomotor policy learning via implicit maximum likelihood estimation,” *ArXiv*, vol. abs/2502.12371, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:276422212>
- [36] A. Bhaskar *et al.*, “Prism: Performer rs-imle for single-pass multisensory imitation learning,” *ArXiv*, vol. abs/2602.02396, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:285271140>
- [37] G. Lee *et al.*, “Implicit maximum likelihood estimation for real-time generative model predictive control,” *ArXiv*, vol. abs/2603.13733, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:286568748>
- [38] Y. Lipman *et al.*, “Flow matching for generative modeling,” *ArXiv*, vol. abs/2210.02747, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:252734897>
- [39] R. Cadène *et al.*, “Lerobot: An open-source library for end-to-end robot learning,” *ArXiv*, 2026. [Online]. Available: <https://api.semanticscholar.org/CorpusID:286082457>
- [40] W. Li *et al.*, “Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification,” *ArXiv*, vol. abs/2508.21046, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:280949804>
- [41] M. Shukor *et al.*, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *ArXiv*, vol. abs/2506.01844, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:279119427>
- [42] A. Khazatsky *et al.*, “Droid: A large-scale in-the-wild robot manipulation dataset,” *ArXiv*, vol. abs/2403.12945, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:268531351>